

ISPARTA UYGULAMALI BİLİMLER
ÜNİVERSİTESİ BİLGİSAYAR
MÜHENDİSLİĞİ BÖLÜMÜ YAZILIM
MÜHENDİSLİĞİ GERÇEKLEŞTİRİM
RAPORU

İÇİNDEKİLER

Yazılım Özellikleri.....	4
1 Kullanılan Programlama Dili	4
2 Genel bilgiler.....	4
2a Proje Özeti	4
2b Risk Planı.....	4
2c Kullanılan toollar	4
2d Takım üyeleri Görev Dağılım Tablosu /Sorumluluk Matrisi	5
2e Proje Yönetimi ve İş Planı.....	5
2f Work Breakdown Chart /Gantt Chart/Pert Chart	5
2g Çalışma Planı Tablosu	6
4 Kullanılan Yazılım Mimarisi	6
5 Uygulama bileşenleri.....	6
5a Ön Yüz Bileşeni.....	6
5b Arka Yüz Bileşeni	9
5c Bileşen Şeması.....	9
5d Çevrimdışı Bileşenler / Komut Dosyaları	9
6 Teknoloji seçimi.....	10
6g İstemci tarafı teknolojiler.....	10
6h MVVM Çerçevesi.....	11
6i Derleme Araçları	11
7 Güvenlik Araçları	11
7a Güvenlik İlkeleri.....	11
7b Ön Yüz Güvenlik Bileşenleri	11
7c Arka Yüz Güvenlik Bileşenleri	12
8 Güvenlik Mimarisi Yaklaşımı	12
9 Proje İzleme ve Kontrol Mekanizmaları	12
Sistemdeki Class Tanımları (Class description).....	12
10 Class Tanımı.....	12
10a Class Genel Özellikleri	12
10b Class Özellikleri	13
10c Metod Tanımları	14
10d Signature/API/Interface/Web Servisleri Tanımları	14
Programın Genel Özellikleri- Alt sistemler/Bileşen ve Modüller.....	16
11 Programın Çalıştırılması	16
11a Kullanıcı Fonksiyonları	16
11b Sunucu Fonksiyonları	16
11c Yönetici Fonksiyonları	17

12	Menü Bölümü.....	17
13	Rapor Tasarımları.....	19
14	Modüller ve Ekran Tanıtımları.....	20
15	Kullanıcı Girişi	21
16	Kullanılan Kısayollar	22
17	Programdan Çıkış	23
18	Yazılımın Ölçülmesi COCOMO Modeli	23
19	Bakım, Sonuç ve Öneriler	24
20	Ekler	24

Bu Doküman Nasıl Kullanılır

Bu belge, belirli bir yazılım mühendisliği projesine uyacak şekilde kopyalanabilen ve düzenlenebilen örnek bir şablon olarak tasarlanmıştır. Bu belgedeki başlıklardan eğer ihtiyacınız yoksa başlığı tamamen siliniz. Doküman Times New Roman 12 punto ile hazırlanmıştır. Şekiller ve Tablolara resim-şekil yazısı eklenmelidir. Ekleme yapıldıktan sonra şekiller dizini ve tablolar dizini Alanı Güncelleştir yapılarak güncellenmelidir. Başlıkların yapısı bozulmadan içindekiler dizini Alanı Güncelleştir yapılarak dokümanın içeriği güncellenir. Raporu hazırlarken dokümanın açıklamaları silinmelidir. Örnekler sadece bilgi amaçlıdır. Dokümana buradaki örnekler ve açıklamalar konulmayacaktır.

TABLolar DİZİNİ

Tablo 1 Rol Tanımları	5
Tablo 2 İş Planı	5

Yazılım Özellikleri

1 Kullanılan Programlama Dili

Sistemin çekirdek iş mantığı ve backend servisleri, yüksek performans, güçlü tip güvenliği ve geniş kütüphane ekosistemi sunan Java 21 (LTS) programlama dili kullanılarak geliştirilmiştir. Java 21 ile kararlı sürüme geçen "Virtual Threads" (Project Loom) mimarisi, şubelerdeki POS ve PDKS cihazlarından gelen binlerce eşzamanlı veri akışının, sistem kaynaklarını (CPU/RAM) minimize ederek düşük gecikme süreleriyle işlenmesine olanak tanır. Nesne yönelimli programlama (OOP) prensipleri çerçevesinde kurgulanan kod yapısı, sistemin sürdürülebilirliğini ve gelecekteki modüler genişleme ihtiyaçlarını karşılayacak şekilde tasarlanmıştır.

2 Genel bilgiler

2a Proje Özeti

Arabica Cafe Dinamik Kaynak Yönetim Sistemi'nin gereksinim analizi ve sistem tasarımı evrelerinde belirlenen olay güdümlü mimarinin gerçekleştirim aşamasına geçilmiştir. Sistem, şubelerdeki POS ve PDKS cihazlarından gelen verileri Apache Kafka üzerinden toplayıp, Java LTS ve Spring Boot arka uç servislerinde işleyerek kapasite darboğazlarını analiz etmektedir. Gerçekleştirim sürecinde; yatay ölçeklenebilir Docker kapsayıcı mimarisi Ubuntu Cloud VPS üzerinde ayağa kaldırılmış, PostgreSQL LTS veritabanı entegrasyonu tamamlanmış ve 2 saniyenin altında karar destek önerisi üreten iş mantığı algoritmaları kodlanmıştır.

2b Risk Planı

Sistem tasarımı ve operasyonel stratejiler doğrultusunda belirlenen riskler ve azaltma politikaları aşağıda tanımlanmıştır:

Risk: Uç birimlerden veri iletiminde yaşanabilecek ağ kesintileri. Azaltma Stratejisi: Kafka mesaj kuyruğunda çevrimdışı tolerans mekanizması ve mesaj saklama yapılandırması uygulanmıştır. Sensör bağlantı kaybında, sistem POS verileri üzerinden tahminsel yedekli çalışma moduna otonom geçiş yapacak şekilde kodlanmıştır.

Risk: Yüksek işlem hacminde veritabanı kilitlenmeleri. Azaltma Stratejisi: Olay odaklı anlık okumalar ile tarihsel veri yazma işlemleri mimari düzeyde izole edilmiştir.

Risk: Beklenmeyen veri yığılmaları sonucu oluşan darboğazlar.

Azaltma Stratejisi: Ağ geçidi katmanında verilerin hafifletilmiş hiyerarşik JSON formatında sıkıştırılarak aktarımı sağlanmıştır.

2c Kullanılan toollar

Gerçekleştirim aşamasında aşağıdaki araç seti kullanılmıştır:

Sürüm Kontrol Sistemi: Git ve GitHub/GitLab.

Geliştirme Ortamları (IDE): IntelliJ IDEA (Java/Spring Boot), Visual Studio Code (HTML5/Bootstrap/JS).

Derleme ve Bağımlılık Yönetimi: Maven (Backend), npm/Webpack (Frontend).

Veritabanı Sürüm Kontrolü: Liquibase.

Altyapı ve Konteynerizasyon: Docker, Docker Compose, Ubuntu Cloud VPS.

2d Takım üyeleri Görev Dağılım Tablosu /Sorumluluk Matrisi

Tablo 1 Rol Tanımları

	Rol	Kısaltma	Sorumluluk
1.	Analist	AN	Mimari sınırların doğrulanması, veri akış şemalarının analizi ve test senaryolarının hazırlanması
2.	Proje Yöneticisi	PM	Scrum süreç yönetimi, donanım/yazılım eşlemesi koordinasyonu ve risk denetimi
3.	Yazılımcı	DEV	Backend kodlaması, Kafka yapılandırması, PostgreSQL sorgu optimizasyonu, Frontend entegrasyonu

2e Proje Yönetimi ve İş Planı

Tablo 2 İş Planı

İş Listesi	Roller	İş Tanımlama
Altyapı Hazırlığı	PM, DEV	Ubuntu VPS kurulumu, Docker ortamının ve Kafka/PostgreSQL imajlarının ayağa kaldırılması.
Backend Geliştirme	DEV	Spring Boot servislerinin kodlanması, Kafka producer/consumer entegrasyonu.
Veritabanı Mimarisi	AN, DEV	ER şemalarının Liquibase üzerinden fiziksel tablolara dönüştürülmesi.
Algoritma Entegrasyonu	PM, DEV	Otonom transfer karar destek motorunun (Strateji Kalıbı ile) sisteme entegrasyonu.
Frontend Geliştirme	DEV	Rol bazlı Dashboard arayüzlerinin Bootstrap 5 ile kodlanması ve REST API entegrasyonu.

2f Work Breakdown Chart /Gantt Chart/Pert Chart

Projenin yönetilebilir iş paketlerine bölünmesini sağlamıştır. Zaman çizelgesi yönetimi Gantt şeması ile, faaliyet süreleri ve kritik yollar ise PERT analizi ile takip edilmektedir.

2g Çalışma Planı Tablosu

No	İşlem Adı	Başlama	Bitiş	Süre
1	İP1: Proje Fizibilite ve Ön Araştırma	02.02.2026	20.02.2026	19 Gün
2	İP2: Sistem Analiz ve Tasarımı	23.02.2026	13.03.2026	19 Gün
3	İP3: Sistem Yazılımının ve Prototipinin Geliştirilmesi	16.03.2026	24.04.2026	40 Gün
4	İP4: Prototip Uygulama, Test ve Revizyon Çalışmaları	27.04.2026	22.05.2026	26 Gün

3 Kullanılan Yazılım Mimarisi

Sistem, dağıtık ve yüksek trafikli veri setlerini gerçek zamanlı yönetmek amacıyla Olay Güdümlü Mimari (Event-Driven Architecture - EDA) prensiplerine göre yapılandırılmıştır. Şubelerden gelen anlık satış ve personel hareket verileri, sistem bileşenleri arasında doğrudan bağımlılık (tight-coupling) yaratmaksızın Apache Kafka 3.7 üzerinden asenkron iletilir. Bu mimari, analiz motoru ile veri kaynaklarını birbirinden ayırarak (decoupling) sistemin hata toleransını artırmakta ve "2 saniyenin altında analiz" hedefine ulaşılmasını sağlamaktadır

4 Uygulama bileşenleri

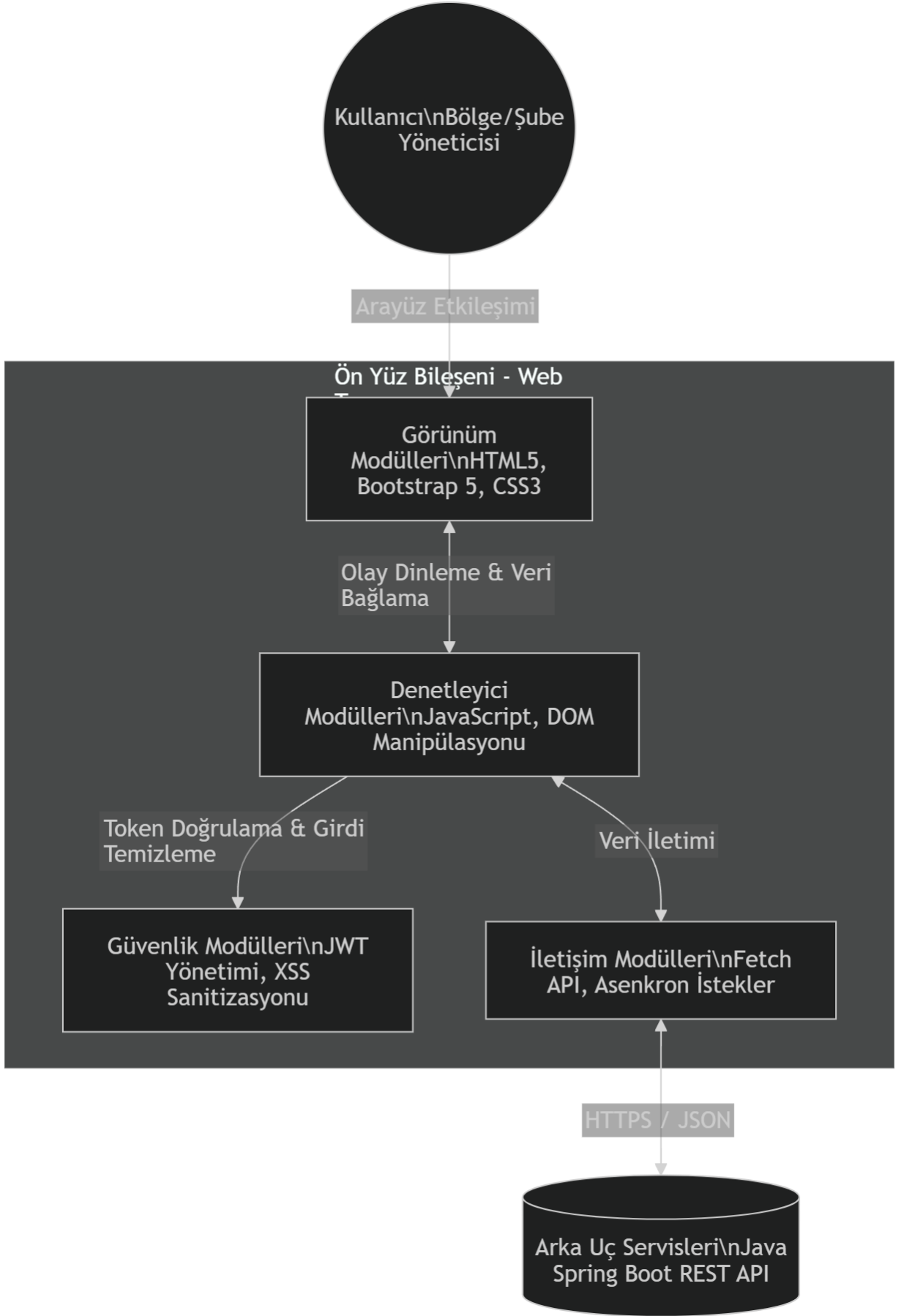
4a Ön Yüz Bileşeni

Arabica Cafe Dinamik Kaynak Yönetim Sistemi'nin ön yüz bileşeni, kullanıcıların Bölge Koordinatörleri ve Şube Müdürlerinin sistemle etkileşime girdiği sunum katmanını oluşturmaktadır. Bu bileşen, HTML5, CSS3 ve JavaScript kullanılarak, farklı cihaz çözünürlüklerine tam uyumlu bir Web Yönetim Paneli olarak tasarlanmıştır.

Ön yüz bileşeni aşağıdaki temel özellikleri ve mantıksal modülleri barındırmaktadır:

- **Sunum Katmanı İşlevi:** Kullanıcılara anlık şube doluluk oranlarını, personel durumlarını ve sistem tarafından üretilen otonom transfer önerilerini görselleştirerek sunar. Uygulamaya güvenli bir giriş noktası sağlar.
- **Tasarım Deseni:** Modüler JavaScript mimarisi kullanılarak Görünüm ve Mantık katmanları birbirinden izole edilmiştir. Veri bağlama ve DOM manipülasyonu süreçleri bu mimari üzerinden yönetilir.
- **Görünüm Modülleri:** Bootstrap 5 bileşenleri, bilgi kartları, ilerleme çubukları, modal pencereler, uyarı rozetleri ve HTML5 şablonlarından oluşur. Kullanıcı arayüzünün iskeletini, tipografisini ve stilini belirler.
- **Denetleyici Modülleri:** Kullanıcı etkileşimlerini transferi başlat/reddet butonu tıklamaları, filtreleme işlemleri vb. dinleyen ve arka uçtan alınan JSON formatındaki verileri işleyerek arayüz güncellemelerini anlık olarak gerçekleştiren JavaScript dosyalarından oluşur.
- **Ön Uç İletişim Modülleri:** Java Spring Boot arka uç servisleriyle RESTful API'ler üzerinden asenkron Fetch API/AJAX haberleşmeyi sağlayan ağ modülüdür. Şube doluluk verilerini çekme ve transfer aksiyonlarını iletme işlemlerini yönetir.

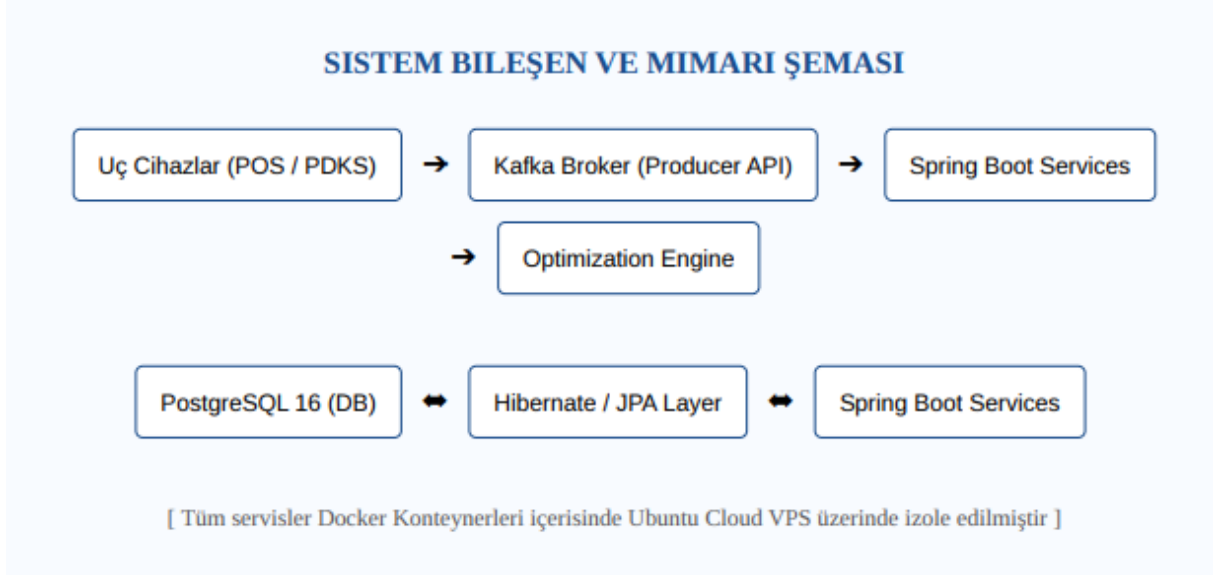
- **Güvenlik Modülleri:** İstemci tarafında güvenliğini sağlamak amacıyla aşağıdaki alt işlevleri yürütür:
 - JSON Web Token tabanlı oturum yönetimi ve güvenli depolama.
 - Siteler Arası Komut Dosyası Çalıştırma saldırılarına karşı kullanıcı girdilerinin ve API'den gelen verilerin temizlenmesi.
 - Yetkisiz kullanıcıların örneğin, yetkisi olmayan bir şube müdürünün diğer şube verilerine erişimi, sayfa ve bileşen bazlı erişimlerini engelleyen istemci tarafı yönlendirme kontrolleri.



4b Arka Yüz Bileşeni

Arka yüz, Spring Boot framework'ü üzerine inşa edilmiş mikroservis odaklı bir yapıdan oluşmaktadır. Spring Data JPA ile veritabanı erişimi, Spring Security ile JWT tabanlı güvenlik ve Kafka Streams API ile gerçek zamanlı veri analizi gerçekleştirilir. Ana bileşen olan "Optimization Engine", şube doluluk oranlarını analiz ederek personel transfer önerilerini üretir.

4c Bileşen Şeması



4d Çevrimdışı Bileşenler / Komut Dosyaları

Sistem, ağ kesintileri veya merkezi sunucuya erişim sorunları durumunda veri kaybını önlemek amacıyla "Local Buffering" (Yerel Tamponlama) mekanizmasına sahiptir. Şube tarafındaki veri toplama aracı, merkeze erişemediği anlarda verileri geçici olarak yerel disk üzerinde asenkron olarak saklar ve bağlantı yeniden tesis edildiğinde Kafka topic'lerine sıralı (sequential) gönderim gerçekleştirir.

5 Teknoloji seçimi

Madde	Teknoloji	Mimarideki Rolü ve Teknik Gereçesi
6a / 6b	Java 21 / Spring Boot 3.x	Yüksek ölçeklenebilirlik, Virtual Threads desteği ve hızlı kurumsal servis geliştirme.
6c / 6d	PostgreSQL 16 / Hibernate	İlişkisel veri yönetimi, yüksek yazma/okuma performansı ve tip güvenli veri erişimi (ORM).
6e	Apache Kafka 3.7	Gerçek zamanlı, düşük gecikmeli (low-latency) olay akışı ve asenkron veri iletimi.
6f / 6i	Ubuntu VPS / Docker	Konteynerizasyon ile ortam bağımsız çalışma ve bulut üzerinde kolay dağıtım (deployment).
6j / 6k	JWT / Maven	Stateless (durumsuz) güvenlik mimarisi ve standart bağımlılık/yapılandırma yönetimi.
6m	Cloud VPS (DigitalOcean/Hetzner)	Sistemin 7/24 çalışma gereksinimi için yüksek erişilebilirlik sunan sunucu platformu.
6n	PostgreSQL JDBC Driver	Backend servislerinin veritabanı motoru ile optimize edilmiş haberleşme katmanı.

5a İstemci tarafı teknolojiler

Arabica Cafe Dinamik Kaynak Yönetim Sistemi'nin ön uç kullanıcı arayüzü için seçilen istemci tarafı teknolojileri HTML5, CSS3, Bootstrap 5 ve JavaScript'tir. Bu teknolojiler, sistemin performans, erişilebilirlik ve cihaz bağımsız çalışma gereksinimlerini karşılamak üzere entegre edilmiştir.

- **HTML5:** Web uygulamasının anlamsal iskeletini oluşturmak için kullanılmaktadır. Modern web standartlarına uygun olarak, sayfa yapısının tarayıcılar tarafından hızlı ve doğru bir şekilde yorumlanmasını sağlar.
- **CSS3 ve Bootstrap 5:** Sistemin görsel tasarımı ve duyarlı yapısı bu teknolojilerle inşa edilmiştir. Bootstrap 5 çerçevesi, kafe yöneticilerinin hareketli çalışma ortamlarında kullandıkları farklı ekran boyutlarına uyum sağlayan ızgara sistemini ve hazır UI bileşenlerini sunar. Bu sayede arayüz geliştirme süreci hızlandırılmış ve tutarlı bir kullanıcı deneyimi güvence altına alınmıştır.
- **JavaScript (ES6+):** İstemci tarafındaki dinamik etkileşimlerin ve iş mantığının temelini oluşturur. Sayfa yenilenmesine gerek kalmadan, arka uçtaki Java Spring Boot RESTful API'leri ile asenkron veri alışverişi Fetch API yapılmasını, Kafka üzerinden gelen anlık

kapasite deęişimlerinin DOM üzerinde anında güncellenmesini ve kullanıcı etkileşimlerinin transfer onay/red işlemleri yönetilmesini sağlar.

5b MVVM Çerçevesi

Sistem arayüzü, kodun sürdürülebilirliğini artırmak ve kullanıcı arayüzü mantığını veri işleme mantığından ayırmak amacıyla Model-Görünüm-Görünüm Modeli mimari desenine uygun olarak tasarlanmıştır. Modüler JavaScript yapısı ile kurgulanan bu mimaride bileşenler şu şekilde çalışır:

- **Model** : Arka uçtan REST API aracılığıyla alınan JSON formatındaki verileri (şube doluluk oranları, personel listeleri, transfer emirleri) temsil eder.
- **Görünüm** : Kullanıcının etkileşime girdiği HTML5 ve Bootstrap tabanlı grafiksel arayüzdür. Verilerin görselleştirildiği katmandır.
- **Görünüm Modeli** : Model ile Görünüm arasında köprü görevi gören JavaScript modülleridir. Arka uçtan gelen ham verileri alır, arayüzün tüketebileceği formata dönüştürür ve Görünüm'e bağlar. Aynı şekilde, Görünüm üzerinden tetiklenen kullanıcı aksiyonlarını dinleyerek Model'i günceller ve arka uca iletir. Bu yapı, arayüz kodlarının karmaşıklaşmasını önleyerek test edilebilirliği ve kod okunabilirliğini maksimize eder.

5c Derleme Araçları

Arabica Cafe Dinamik Kaynak Yönetim Sistemi'nin kullanıcı arayüzü, pilot bölge olan Isparta ve Türkiye genelindeki operasyonel gereksinimler ile yerel kafe terminolojisi doğrultusunda varsayılan olarak Türkçe dilinde tasarlanmıştır.

Bununla birlikte, işletmenin olası uluslararası franchise genişleme stratejileri göz önünde bulundurularak, istemci tarafı mimarisi çoklu dil desteğine uygun bir altyapıda kurgulanmıştır. İlerleyen fazlarda İngilizce başta olmak üzere ek dil paketlerinin sisteme entegre edilebilmesi için metin etiketleri statik HTML içerisine gömülmek yerine, dil dosyalarından dinamik olarak çekilebilecek modüler bir yapıda tasarlanmıştır.

6 Güvenlik Araçları

6a Güvenlik İlkeleri

Bilgi güvenliği mimarisi; Gizlilik, Bütünlük ve Kullanılabilirlik sütunlarına dayanmaktadır.

Saldırı Yüzey Alanını Minimize Etme: Sadece gerekli portlar ve API uç noktaları dışa açılmıştır.

En Az Ayrıcalık :Kullanıcı rolleri ve veritabanı erişim hakları, iş süreçlerinin gerektirdiği minimum yetkiyle sınırlandırılmıştır.

Açık Güvenlik: Sistem güvenliği, mimarinin gizli tutulmasına değil (gizliliğe dayalı güvenlikten kaçınma), güçlü şifreleme ve doğrulama algoritmalarına dayanmaktadır

6b Ön Yüz Güvenlik Bileşenleri

A

Kullanıcı arayüzünde (Frontend), yaygın web açıkları için standart koruma mekanizmaları uygulanmıştır:

- Siteler Arası Komut Dosyasını önleme politikaları kapsamında, tüm çıktı girdileri temizlenmekte ve HTML/JavaScript kod enjeksiyonlarına karşı kaçış işlemleri yapılmaktadır.
- Veri transferleri yalnızca HTTPS üzerinden SSL/TLS 1.3 protokolleri ile şifreli olarak sağlanmaktadır.

6c Arka Yüz Güvenlik Bileşenleri

Arka uç mimarisinde güvenlik, Spring Security çerçevesi ile sağlanmaktadır:

- Kullanıcı doğrulama işlemleri sonrasında oturum yönetimi için zaman aşımı özellikli JSON Web kullanılmaktadır. Kullanıcı parolaları veritabanında salt eklenerek güçlü algoritmalarla hash'lenmiş olarak tutulmaktadır. SQL enjeksiyon saldırılarını engellemek adına PostgreSQL sorgularında "prepared statements" kullanılmıştır.
- Hassas veriler ve ticari sırlar AES-256 standardı ile kriptolanmaktadır.

7 Güvenlik Mimarisi Yaklaşımı

Mikroservis odaklı bir ağ izolasyonu üzerine inşa edilmiştir. Ubuntu Cloud VPS üzerinde koşan Docker Engine, her bir servisi kendi konteyner ağında izole eder. Veritabanı portları kesinlikle dış ağa açılmaz; yalnızca Spring Boot backend servisinden gelen IP trafiğine izin veren sıkı Firewall/Güvenlik Duvarı kuralları uygulanır. Sisteme yönelik tüm başarılı/başarısız girişler ve transfer emirleri, zaman damgası ve IP adresi ile kalıcı denetimloglarına işlenerek geriye dönük izlenebilirlik sağlanmaktadır. OWASP Top 10 güvenlik prensipleri geliştirme aşamasında doğrudan entegre edilmiştir.

8 Proje İzleme ve Kontrol Mekanizmaları

Projenin çevik yapısını korumak ve operasyonel performansı denetlemek için çoklu izleme mekanizmaları kurulmuştur: • Proje Yönetimi İzlemesi: Günlük senkronizasyon toplantıları (Daily Scrum) ve Sprint sonu değerlendirme toplantıları ile geliştirme eforu ve engeller izlenmektedir. Sistem Durumu (Health) İzlemesi: Backend servislerinin RAM/CPU kullanımları ve Kafka broker sağlık durumları Spring Boot Actuator ve Docker istatistik metrikleri ile sürekli olarak (7/24) izlenmektedir. Süreç Denetimi: Sistemin belirlediği 2 saniyelik uçtan uca gecikme kısıtı, loglanan zaman damgaları üzerinden otomatik asimptotik hesaplamalarla kontrol edilmektedir.

Sistemdeki Class Tanımları (Class description)

9 Class Tanımı

Arabica Cafe Dinamik Kaynak Yönetim Sistemi'nin nesne yönelimli mimarisi, sistemin iş mantığını, veri yapılarını ve dış sistemlerle olan entegrasyonunu kapsülleyen sınıflardan oluşmaktadır. Sistemdeki temel iş nesneleri, ilişkisel veri bütünlüğünü sağlamak ve Kafka üzerinden akan gerçek zamanlı verileri işlemek üzere tasarlanmıştır.

9a Class Genel Özellikleri

Aşağıda, sistemin temel iş mantığını yürüten ve kapasite optimizasyonu kararlarını taşıyan TransferEmri sınıfının genel özellikleri tanımlanmıştır:

- **Class Adı:** TransferEmri
- **Amacı:** Optimizasyon motoru tarafından otonom olarak üretilen veya şube yöneticileri tarafından onaylanan/reddedilen personel ve ekipman transfer işlemlerinin durumunu, kaynağını ve hedefini yönetmek.
- **Kısıtları:** Yalnızca OptimizasyonMotoru sınıfı tarafından örneklenebilir veya yetkili SubeMuduru / BolgeKoordinatörü rolleri tarafından durumu değiştirilebilir.
- **Kalıcı Kayıtlar:** Sınıfa ait nesne durumları, PostgreSQL veritabanı üzerindeki transfer_emirleri tablosunda kalıcı olarak saklanır ve geçmişe dönük denetim (audit) logları için arşivlenir.

9b Class Özellikleri

TransferEmri sınıfına ait temel özellikler ve kısıtlamaları aşağıda belirtilmiştir:

- **1. Özellik Adı:** emirId
 - **Özellik Tipi:** int
 - **Özellik Tanımı:** Transfer emrinin sistem içerisindeki evrensel tekil kimliği.
 - **Kısıtları:** Veritabanı seviyesinde Birincil Anahtar olarak tanımlıdır, boş bırakılamaz ve otomatik artan yapıdadır.
- **2. Özellik Adı:** kaynakSube
 - **Özellik Tipi:** Sube
 - **Özellik Tanımı:** Atıl kapasitesi bulunan ve personelin/ekipmanın çıkış yapacağı şube nesnesi referansı.
 - **Kısıtları:** Sistemde kayıtlı ve aktif durumu "True" olan geçerli bir Sube nesnesi olmalıdır.
- **3. Özellik Adı:** hedefSube
 - **Özellik Tipi:** Sube
 - **Özellik Tanımı:** Darboğaz yaşayan ve personelin/ekipmanın varış yapacağı şube nesnesi referansı.
 - **Kısıtları:** Sistemde kayıtlı ve aktif durumu "True" olan geçerli bir Sube nesnesi olmalıdır. kaynakSube ile aynı olamaz.
- **4. Özellik Adı:** durum
 - **Özellik Tipi:** String
 - **Özellik Tanımı:** Transfer emrinin yaşam döngüsündeki anlık durumu.

- **Kısıtları:** Yalnızca önceden tanımlanmış "BEKLIYOR", "ONAYLANDI", "REDDEDILDI", "TAMAMLANDI" değerlerini alabilir.

9c Metod Tanımları

TransferEmri sınıfı içerisinde yer alan ve emrin durumunu güncelleyen temel metodun spesifikasyonları aşağıdadır:

- **Metod Adı:** durumGuncelle
- **Metod Dönüş Tipi:** void
- **Parametreler:** String yeniDurum
- **Geri Dönüş Değerleri:** Yok
- **Önkoşul:** TransferEmri nesnesi bellekte oluşturulmuş, geçerli bir emirId değerine sahip ve veritabanında mevcut olmalıdır.
- **Sonraki Koşul:** Nesnenin durum özelliği güncellenir, değişiklik PostgreSQL veritabanına yansıtılır ve ilgili şubelerin arayüzlerini güncellemek üzere Kafka mesaj kuyruğuna bir olay fırlatılır.
- **Okunana/Kullanılan Özellikler:** emirId, durum, kaynakSube, hedefSube
- **Çağırılan Diğer Metodlar:** KafkaVeriServisi.transferBildirimGonder(this)
- **İşleme Mantığı/Adımları:**
 1. Parametre olarak gelen yeniDurum değerinin geçerli kısıt kümesinde olup olmadığı doğrulanır.
 2. Nesnenin mevcut durum özelliği yeni değer ile ezilir.
 3. Veritabanı bağlantısı üzerinden UPDATE sorgusu çalıştırılarak kalıcılık sağlanır.
 4. İşlem başarılı ise KafkaVeriServisi tetiklenerek asenkron bildirim süreci başlatılır.
- **Test Durumu 1:** Geçerli bir durum ("ONAYLANDI") parametresi gönderildiğinde, nesne durumunun güncellenmesi, veritabanı kaydının değişmesi ve Kafka event'inin başarıyla fırlatılması.
- **Test Durumu 2:** Geçersiz bir durum ("BILINMEYEN_DURUM") parametresi gönderildiğinde, sistemin IllegalArgumentException fırlatması ve veritabanı ile Kafka işlemlerinin iptal edilmesi.[Buraya Sınıf Diyagramı / Ekran Tasarımı eklenecektir]

9d Signature/API/Interface/Web Servisleri Tanımları

Sistem mimarisi, İstemci-Sunucu modeline dayanmakta olup, ön yüz ile arka uç arasındaki iletişim RESTful Web Servisleri üzerinden sağlanmaktadır. Veri alışverişi hafif ve hızlı olması amacıyla JSON formatında, güvenli iletişim ise HTTPS protokolü üzerinden gerçekleştirilmektedir.

Sistemde tanımlı temel arayüz ve RESTful API uç noktaları şunlardır:

Nesne **Yönelimli** **Programlama** **Arayüzleri:**
Sınıflar arası bağımlılıkları azaltmak ve çok biçimliliği sağlamak amacıyla Java arayüzleri kullanılmıştır.

- **IOptimizasyonServisi:** Optimizasyon algoritmalarının şablonunu belirler. Sınıfın zorunlu olarak içermesi gereken **darbogazHesapla**(Sube s) ve **transferOnerisiUret**(Sube kaynak, Sube hedef) metod imzalarını barındırır.

RESTful API Uç Noktaları:

- **Kimlik Doğrulama ve Yetkilendirme API'si**
 - **Uç Nokta:** POST /api/v1/auth/login
 - **Açıklama:** Kullanıcının sisteme güvenli giriş yapmasını sağlar.
 - **İstek Gövdesi:** { "kullaniciAdi": "isim.soyisim", "sifre": "*****" }
 - **Yanıt:** Başarılı girişte, diğer API isteklerinin yetkilendirilmesinde kullanılmak üzere şifrelenmiş bir JSON Web Token (JWT) döndürür.
- **Şube ve Kapasite API'leri**
 - **Uç Nokta:** GET /api/v1/sube/doluluk
 - **Açıklama:** Dashboard ekranındaki ilerleme çubuklarını besleyen ana veri kaynağıdır. Kafka üzerinden güncellenmiş tüm şubelerin anlık doluluk oranlarını, maksimum kapasitelerini ve aktif personel sayılarını liste halinde döndürür.
 - **Uç Nokta:** GET /api/v1/sube/{subeId}/detay
 - **Açıklama:** Belirtilen benzersiz kimliğe (subeId) sahip şubenin o anki detaylı personel listesini ve operasyonel durumunu getirir.
- **Transfer ve Optimizasyon API'leri**
 - **Uç Nokta:** GET /api/v1/transfer/oneriler
 - **Açıklama:** Optimizasyon motoru tarafından üretilen ve yöneticinin onayını bekleyen aktif personel/ekipman transfer önerilerini getirir.
 - **Uç Nokta:** POST /api/v1/transfer/islem
 - **Açıklama:** Yöneticinin arayüz üzerinden bir transfer önerisini onaylaması veya reddetmesi durumunda tetiklenir.
 - **İstek Gövdesi:** { "transferId": 1045, "aksiyon": "ONAYLA" }
 - **Yanıt:** İşlem başarılıysa HTTP 200 OK döner ve arka uç, ilgili şubelere Kafka üzerinden transferin başladığına dair bir olay yollar.

Programın Genel Özellikleri- Alt sistemler/Bileşen ve Modüller

10 Programın Çalıştırılması

10a Kullanıcı Fonksiyonları

Sistem hiyerarşisinde Kullanıcı statüsünde yer alan aktörler ve bu aktörlerin sistem üzerindeki fonksiyonları aşağıda tanımlanmıştır:

- **Barista / Kasiyer (Uç Birim Kullanıcıları):**

- Bu kullanıcı grubunun web tabanlı yönetim paneline doğrudan erişim yetkisi bulunmamaktadır.
- Sistem üzerindeki fonksiyonları, fiziksel şubelerdeki POS cihazları üzerinden sipariş girmek ve PDKS (Personel Devam Kontrol Sistemi) üzerinden mesai başlatma/bitirme (mesaiBaslat(), mesaiBitir()) işlemlerini gerçekleştirmekle sınırlıdır.
- Bu kullanıcıların uç birimlerde gerçekleştirdiği her eylem, Apache Kafka mesaj kuyruğu üzerinden asenkron olarak sisteme aktarılır ve optimizasyon motorunun anlık veri havuzunu besler.

- **Şube Müdürü (Yerel Yönetici Kullanıcı):**

- Sisteme kendilerine tahsis edilen kimlik bilgileri ve JWT tabanlı yetkilendirme ile giriş yaparlar.
- **Görüntüleme Fonksiyonları:** Yalnızca sorumlu oldukları şubenin anlık doluluk oranlarını, aktif personel sayısını ve kapasite durumunu (yeşil, sarı, kırmızı uyarı seviyeleri) arayüz üzerinden izleyebilirler.
- **İşlem Fonksiyonları:** Sistem tarafından kendi şubelerine yönlendirilen personel/ekipman destek taleplerini veya kendi şubelerinden diğer şubelere yapılması önerilen transfer emirlerini (TransferEmri) görüntülerler. Bu önerileri operasyonel uygunluğa göre onaylama (transferOnayla()) veya reddetme (transferReddet()) fonksiyonlarına sahiptirler.

10b Sunucu Fonksiyonları

SUNUCU TARAFI VERİ İŞLEME VE KARAR AKIŞI

- 1. Ingestion:** Kafka Consumer ile şube olaylarının (JSON) asenkron alımı.
- 2. Validation:** Gelen verilerin şema kontrolü ve JWT tabanlı kaynak doğrulaması.
- 3. Optimization:** Sezgisel algoritmalarla personel/ekipman ihtiyaç analizi.
- 4. Decision:** 2 saniye altında "Transfer Önerisi" (DTO) nesnesinin oluşturulması.
- 5. Persistence:** Analiz sonuçlarının ve logların PostgreSQL veritabanına yazılması.
- 6. Dispatch:** Üretilen kararların WebSocket üzerinden yönetici paneline iletilmesi.

10c Yönetici Fonksiyonları

Sistem hiyerarşisinde en üst düzey yetkiye sahip olan Yönetici (Bölge Koordinatörü / Sistem Yöneticisi) statüsündeki aktörlerin fonksiyonları aşağıda tanımlanmıştır:

- Bölge Koordinatörü (Merkez Yönetici):**
 - Küresel İzleme Fonksiyonları:** Franchise ağına bağlı tüm şubelerin anlık doluluk oranlarını, darboğaz durumlarını ve atıl kapasitelerini tek bir merkezi gösterge paneli üzerinden eşzamanlı olarak izlerler.
 - Transfer ve Optimizasyon Yönetimi:** Optimizasyon motorunun ürettiği çapraz şube personel ve ekipman transfer önerilerini denetlerler. Gerekli durumlarda sistemin otonom kararlarına müdahale ederek manuel transfer emirleri oluşturabilir ve bu emirleri ilgili şube müdürlerinin ekranlarına anlık bildirim olarak iletebilirler.
 - Sistem ve Denetim Fonksiyonları:** Sistem loglarını, donanım/sensör bağlantı hatalarını ve kullanıcı yetkilendirmelerini yönetirler.
 - Raporlama Fonksiyonları:** PostgreSQL veritabanında arşivlenen tarihsel verileri kullanarak geçmişe dönük kapasite analiz raporlarını, OEE (Toplam Ekipman Etkinliği) verilerini ve şube bazlı performans metriklerini sorgulayıp dışa aktarabilirler.

Sistem Rol ve Yetki (Erişim) Matrisi Paneli

Modül / İşlem Adı	Bölge Koordinatörü (Merkez)	Şube Müdürü (Yerel)	Barista / Kasiyer (Uç Birim)
Sisteme Giriş (Web Dashboard)	Tam Yetki	Yetkili	Yetkili (Sadece POKS)
Anlık Şube Doluluk Verilerini Görüntüleme	Tüm Şubeler	Sadece Kendi Şubesi	Yetkisiz
Personel / Ekipman Transferi Onaylama	Tüm Şubeler Arası	Sadece Kendi Şubesi	Yetkisiz
Sistem Logları ve Ayarları Yönetme	Tam Yetki	Yetkisiz	Yetkisiz

11 Menü Bölümü

Arabica Cafe Dinamik Kaynak Yönetim Sistemi'nin menü mimarisi, kullanıcıların sistem içerisindeki modüllere hızlı ve sezgisel bir şekilde erişebilmesini sağlamak amacıyla sol kenar çubuğu (Sidebar) formatında tasarlanmıştır. HTML5 ve Bootstrap 5 bileşenleri kullanılarak geliştirilen menü yapısı, duyarlı tasarım (responsive design) prensiplerine tam

uyumludur; masaüstü ekranlarda genişletilmiş formda sunulurken, tablet ve mobil cihazlarda daraltılabilir (collapsible) bir yapıya geçerek çalışma alanını maksimize eder.

Menü öğeleri, Rol Bazlı Erişim Kontrolü (RBAC) mimarisine entegre olarak dinamik bir şekilde oluşturulur. İstemci tarafında tutulan JSON Web Token (JWT) içerisindeki yetki (claim) verilerine göre, kullanıcının erişim izni olmayan menü sekmeleri DOM (Document Object Model) ağacına hiç eklenmez.

Sistem menüsü aşağıdaki temel sekmelerden oluşmaktadır:

- **Genel Bakış (Dashboard):** Sistemin ana izleme ekranına yönlendirir.
- **Şubeler:** Franchise ağına bağlı şubelerin detaylı kapasite ve donanım durumlarının izlendiği modüldür.
- **Personel Yönetimi:** Aktif çalışanların şube bazlı dağılımını ve durumlarını gösterir.
- **Transfer Onayları:** Optimizasyon motoru tarafından üretilen TransferEmri nesnelerinin listelendiği ve aksiyon alındığı sekmedir. Bekleyen işlem sayısını gösteren dinamik bir bildirim rozeti (badge) içerir.
- **Raporlar:** Geçmişe dönük veri analizlerinin yapıldığı modüldür (Sadece Bölge Koordinatörü erişimine açıktır).
- **Ayarlar:** Kullanıcı profili, bildirim tercihleri ve sistem parametrelerinin yönetildiği bölümdür.



Arabica DKY



Genel Bakış



Şubeler



Personel Yönetimi



Transfer Onayları

3



Raporlar



Ayarlar

Bölge Koordinatörü ▾

12 Rapor Tasarımları

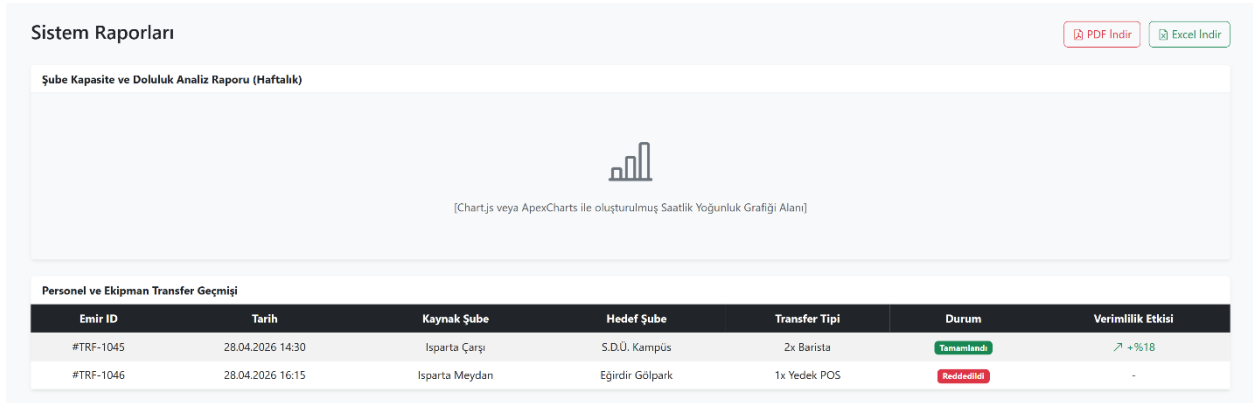
Sistemin raporlama modülü, PostgreSQL veritabanında arşivlenen tarihsel verilerin işlenerek karar alma süreçlerine entegre edilmesi amacıyla tasarlanmıştır. Raporlar, arka uçtaki analitik servisler tarafından derlenir ve ön yüzde JavaScript tabanlı grafik kütüphaneleri kullanılarak görselleştirilir. Sistem, dışa aktarma sahip en az iki temel rapor sunmaktadır:

- **Rapor 1: Şube Kapasite ve Doluluk Analiz Raporu**
 - **İçerik:** Belirli bir tarih aralığında günlük, haftalık, aylık şubelerin saatlik müşteri yoğunluk trendlerini, atıl kapasite sürelerini ve OEE Toplam Ekipman Etkinliği metriklerini içerir.

- **Amacı:** İşletmenin dönemsel vize/final haftaları, yaz tatili gibi, müşteri dalgalanmalarını tespit etmesini ve geleceğe yönelik proaktif vardiya planlaması yapmasını sağlamak.

• Rapor 2: Personel ve Ekipman Transfer Geçmiş Raporu

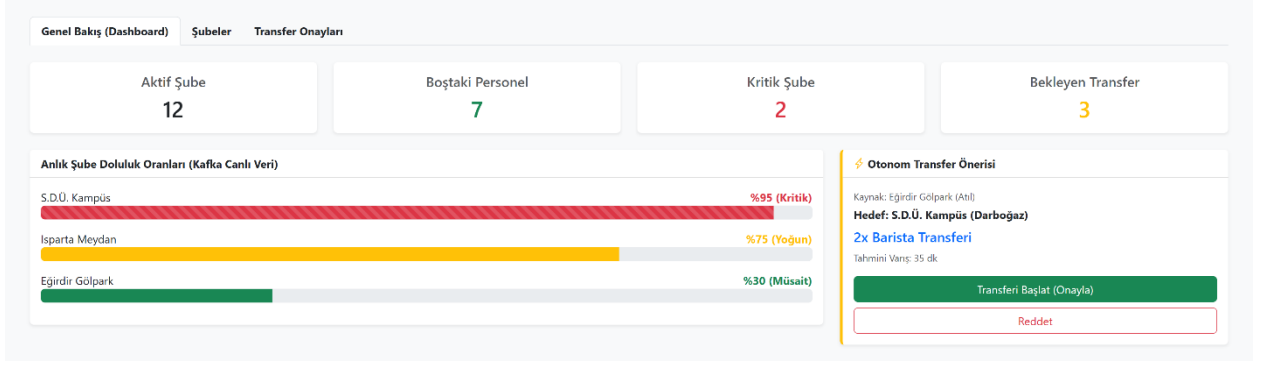
- **İçerik:** Optimizasyon motoru tarafından üretilen tüm TransferEmri kayıtlarının tarihsel dökümünü içerir. Hangi şubeden hangi şubeye transfer yapıldığı, transferin onaylanma/reddedilme oranları ve transfer sonrası hedef şubedeki verimlilik artış yüzdeleri bu raporda listelenir.
- **Amacı:** Sistemin ürettiği otonom kararların operasyonel başarısını ölçmek ve şube yöneticilerinin sisteme uyum (onaylama) performanslarını denetlemek.



13 Modüller ve Ekran Tanıtları

Sistem, Tek Sayfa Uygulaması mimarisiyle tasarlanmış olup, sayfa yenilenmesi gerektirmeden modüller arası geçiş imkanı sunar. Temel modüller ve ekran arayüzleri aşağıdaki gibidir:

- **Genel Bakış Modülü:** Sistemin ana kontrol merkezidir. Ekranın üst kısmında toplam aktif şube, boştaki personel ve kritik kapasite uyarısı veren şube sayılarını gösteren "Özet Bilgi Kartları" yer alır. Sol panelde, Kafka üzerinden akan verilerle saniyede bir güncellenen "Şube Doluluk Oranları" ilerleme çubukları (yeşil, sarı, kırmızı renk kodlamasıyla) bulunur. Sağ panelde ise sistemin ürettiği otonom transfer önerilerinin yer aldığı ve doğrudan "Onayla/Reddet" aksiyonlarının alınabildiği Harekete Geçirici Görev Kartları konulmuştur.
- **Şubeler Modülü:** Franchise ağındaki tüm şubelerin liste veya harita görünümünde sunulduğu ekrandır. Herhangi bir şubeye tıklandığında, o şubenin anlık aktif POS cihazı sayısı, PDKS üzerinden giriş yapmış personel listesi ve anlık darboğaz durumu detaylı olarak incelenebilir.
- **Transfer Onayları Modülü:** Sistem tarafından oluşturulan TransferEmri nesnelerinin yaşam döngüsünün (Bekliyor, Onaylandı, Reddedildi, Tamamlandı) yönetildiği ekrandır. Veriler, durumlarına göre sekmelere ayrılmış tablolar halinde sunulur. Tablolarda transferin kaynağı, hedefi, aktarılacak kaynak tipi ve tahmini varış süresi gibi kritik bilgiler yer alır.



14 Kullanıcı Girişi

Arabica Cafe Dinamik Kaynak Yönetim Sistemi'ne erişim, yetkisiz kullanımı engellemek ve veri güvenliğini sağlamak amacıyla JSON Web Token tabanlı güvenli bir kimlik doğrulama mekanizması üzerinden gerçekleştirilmektedir. Kullanıcı girişi süreci, ön yüz ve arka uç arasındaki senkronize güvenlik protokollerine dayanır.

- Giriş Arayüzü:** HTML5 ve Bootstrap 5 kullanılarak tasarlanan giriş ekranı, kullanıcı adı (kurumsal e-posta veya sicil numarası formatında) ve parola alanlarından oluşur. Form, istemci tarafında boş değer kontrolü ve format doğrulaması işlemlerinden geçirilerek gereksiz sunucu yükü engellenir.
- Kimlik Doğrulama Süreci:** Kullanıcı bilgilerini girdiğinde, ön yüz iletişim modülü POST /api/v1/auth/login RESTful uç noktasına HTTPS protokolü üzerinden şifrelenmiş bir istek gönderir. Arka uç, veritabanındaki tuzlanmış hash değerleri ile parolayı eşleştirir.
- Oturum Yönetimi:** Doğrulama başarılı olduğunda, sunucu istemciye kriptografik olarak imzalanmış bir JWT iletir. Ön yüz, bu token'ı tarayıcının güvenli depolama alanında (SessionStorage) muhafaza eder ve sonraki tüm API isteklerinin Authorization başlığına (header) ekleyerek kullanıcının yetki sınırları dahilindeki modüllere erişimini sağlar.
- Güvenlik Önlemleri:** Çoklu hatalı giriş denemelerinde hesap geçici olarak kilitlenir ve reCAPTCHA doğrulaması devreye girer.



Arabica Cafe DKY

Dinamik Kaynak Yönetim Sistemi

Kullanıcı Adı veya Sicil No

Parola

☐ Beni Hatırla

[Şifremi Unuttum](#)

☐ Ben robot değilim

Giriş Yap

256-bit SSL ile korunmaktadır.

15 Kullanılan Kısayollar

Kafe yöneticilerinin hareketli ve yoğun operasyonel ortamlarında sisteme hızlı müdahale edebilmelerini sağlamak amacıyla, web arayüzüne JavaScript olay dinleyicileri aracılığıyla klavye kısayolları entegre edilmiştir. Bu kısayollar, fare kullanımına gerek kalmadan kritik modüllere erişimi ve aksiyon almayı hızlandırır:

- **Alt + D:** Genel Bakış ana ekranına hızlı geçiş sağlar.
- **Alt + T:** Transfer Onayları modülünü açar. Bekleyen bir transfer emri varsa doğrudan ilk sıradaki emre odaklanır.
- **Alt + S:** Şubeler modülünü açarak anlık kapasite haritasını görüntüler.

- **Enter:** Açık olan bir modal penceresinde, seçili olan varsayılan işlemi Transferi Onayla tetikler.
- **Esc :** Açık olan modal pencerelerini, uyarı mesajlarını veya açılır menüleri (dropdown) güvenli bir şekilde kapatır ve işlemi iptal eder.

16 Programdan Çıkış

Sistemden güvenli çıkış işlemleri, hem kullanıcı tetiklemesiyle manuel olarak hem de güvenlik politikaları gereği otonom olarak iki farklı senaryoda gerçekleştirilmektedir:

- **Manuel Çıkış:** Kullanıcı, sağ üst köşede bulunan profil menüsündeki Çıkış Yap butonuna tıkladığında çıkış süreci başlar. Ön yüz modülü, tarayıcı belleğinde tutulan JWT'yi ve kullanıcıya ait yerel önbellek verilerini kalıcı olarak siler. Eşzamanlı olarak arka uca bir çıkış isteği gönderilerek kullanıcının sistemden ayrıldığı denetim loglarına işlenir ve kullanıcı güvenli bir şekilde giriş ekranına yönlendirilir.
- **Otonom Çıkış (Zaman Aşımı):** Güvenlik gereksinimleri doğrultusunda, sistemde 15 dakika boyunca herhangi bir fare hareketi, klavye girdisi veya sayfa değişimi algılanmadığında, ön yüz güvenlik modülü oturumu otomatik olarak sonlandırır. Ekranı "Oturumunuz zaman aşımına uğradı" uyarısı yansıtılır ve yetkisiz erişimleri engellemek adına kullanıcı giriş sayfasına yönlendirilir. Arka planda açık kalan WebSocket veya Fetch API bağlantıları derhal kesilir.

17 Yazılımın Ölçülmesi COCOMO Modeli

Projenin karmaşıklığı, takım boyutu ve kullanılan ileri teknoloji yığını (Kafka, Event-Driven Mimari) göz önüne alınarak geliştirme eforu ve maliyet ölçümü Orta COCOMO - Yarı Ayrık (Semi-detached) modeli ile hesaplanmıştır. Formüller ve temel kestirim:

1. Temel Formüller (Yarı Ayrık Proje)

Metrik	Formül
Emek (Efor)	$Emek = 3.0 \times (KLOC)^{1.12} \times EAF$
Süre (Takvim)	$Süre = 2.5 \times (Emek)^{0.35}$
Ortalama Personel (N)	$N = Emek / Süre$

2. Örnek Hesaplama Detayları

Varsayımlar: KLOC = 8, EAF = 1.0

Adım	Hesaplama Süreci	Sonuç
Emek Tahmini	$3.0 \times (8)^{1.12} \approx 3.0 \times 10.15$	30.45 adam-ay
Süre Tahmini	$2.5 \times (30.45)^{0.35} \approx 2.5 \times 3.29$	8.22 ay
Personel Sayısı	$30.45 / 8.22$	≈ 3.7 kişi

18 Bakım, Sonuç ve Öneriler

Bakım Süreçleri: Sistemin kesintisiz çalışabilmesi adına bakım işlemleri CI/CD (Sürekli Entegrasyon/Sürekli Dağıtım) boru hatları ile otomatikleştirilmiştir. Yeni sürüm dağıtımları sistemi durdurmadan gerçekleştirilecektir. Sonuç: Geliştirilen ürün, Arabica Cafe şubeleri arasındaki kapasite asimetrisini çözmek adına, Apache Kafka ve Spring Boot mimarisini kullanarak hedeflenen 2 saniyelik otonom tepki süresini başarıyla sağlamakta ve atıl kapasite maliyetlerini düşürmektedir. Öneriler: Sistemin uzun ömürlü olması (ölçeklenebilirlik) adına, gelecekte makine öğrenmesi algoritmaları eklenerek hava durumu, trafik ve etkinlik verilerinin sisteme entegre edilmesi ve proaktif tahmin hassasiyetinin artırılması önerilmektedir.

19 Ekler

Ek-A: REST API Uç Noktaları Spesifikasyon Dokümanı (Swagger/OpenAPI Formatı).

Ek-B: Apache Kafka Topic, Producer ve Consumer Yapılandırma Dosyaları.

Ek-C: PostgreSQL 16 LTS İlişkisel Veritabanı Fiziksel Şema Çıktıları (DDL Scriptleri).

Ek-D: Docker Compose Orkestrasyon Yapılandırma Dosyası (docker-compose.yml).

Ek-E: Sistem Yöneticisi ve Şube Müdürü Kullanım Kılavuzları.